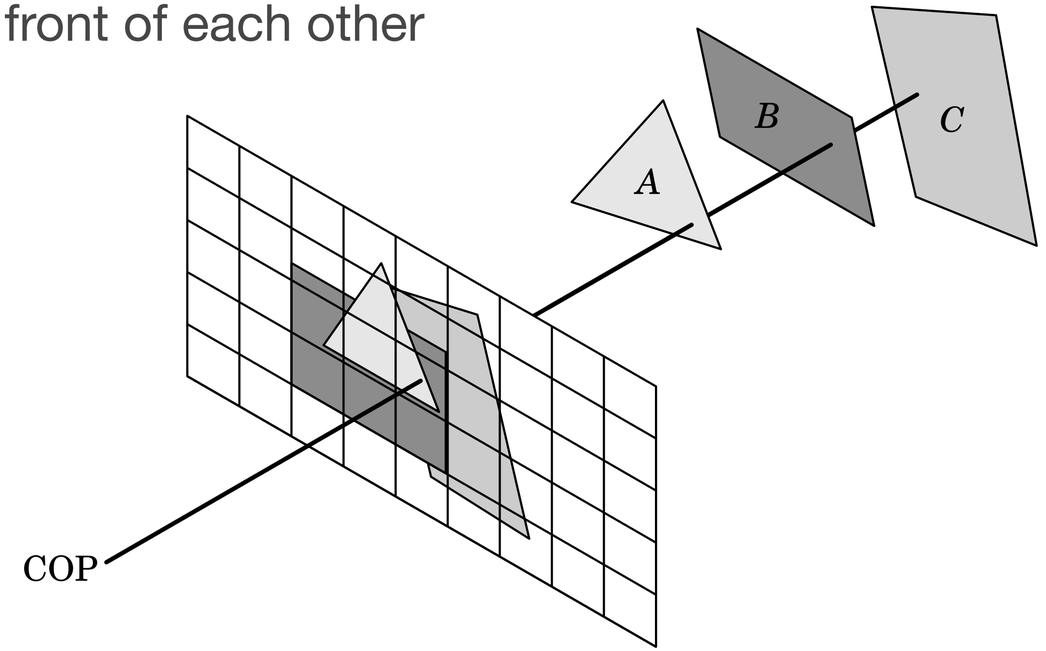
Visibility

e.g. in Assassin's Creed I, 2007

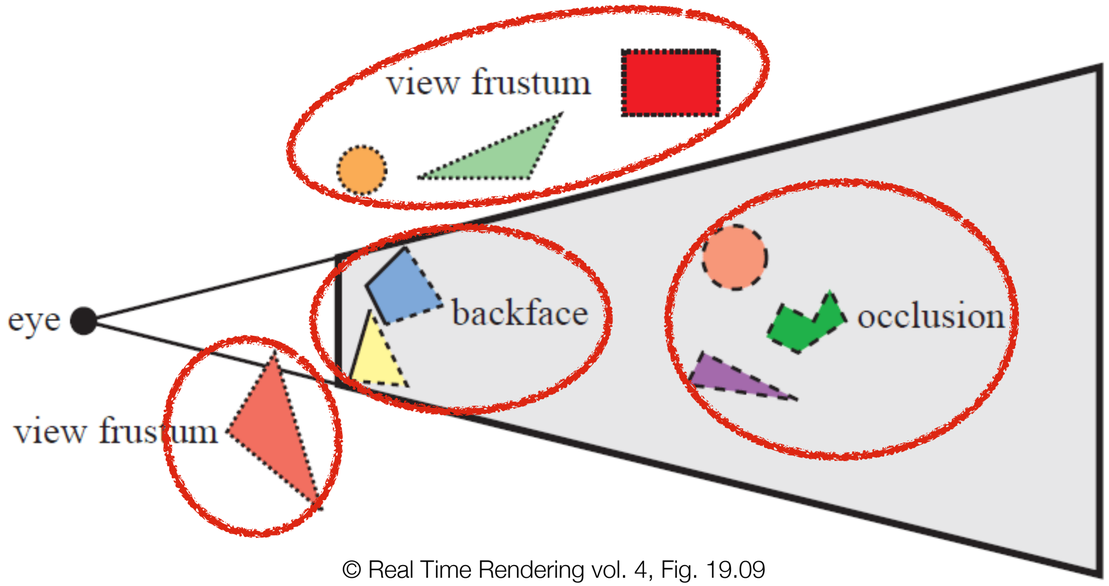
Visibility Problem

- Identify which objects or individual geometric elements, polygons, of surfaces are visible or occluded from a given viewpoint
  - fundamental problem of 3D computer graphics
- Visibility defines what is inside the view frustum in in front of each other
  - a 3D geometry intersection problem (culling)
  - a layering sorting problem
- Solving visible occlusion is a.k.a.
  - Hidden Surface Determination
  - Hidden Surface Removal
  - Visible Surface Determination



© [A00] Figure 7.28

Culling

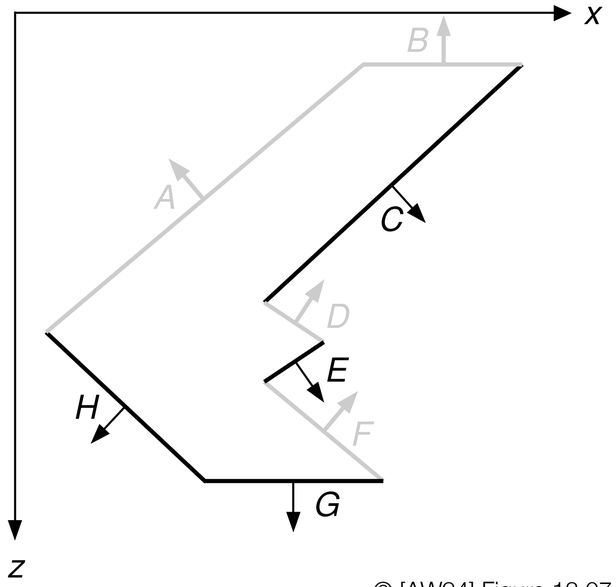


© Real Time Rendering vol. 4, Fig. 19.09

- Identify primitives which cannot be visible and can be reject the before processing and rendering
  - reduces load on the graphics subsystem
- View frustum culling (clipping)
  - reject objects or primitives outside of the view frustum
  - clip as needed if element is intersecting the view frustum
- Back-face culling
  - remove one-sided polygons if viewed from the back/ inside
- Occlusion culling
  - avoid rendering primitives that are occluded by other objects

Back-Face Culling

- Front-facing if viewer can see the visible side of a polygon
  - normal is oriented towards the viewer
- Use sign of dot-product between normal and vector to viewpoint
  - in parallel-projection canonical view volume back-facing
    simply determined by the normal's negative z-component
- Polyhedra and closed manifold surface meshes enclose a solid volume
  - all back-facing polygons are fully occluded by front-facing
  - around 50% of all faces may be culled



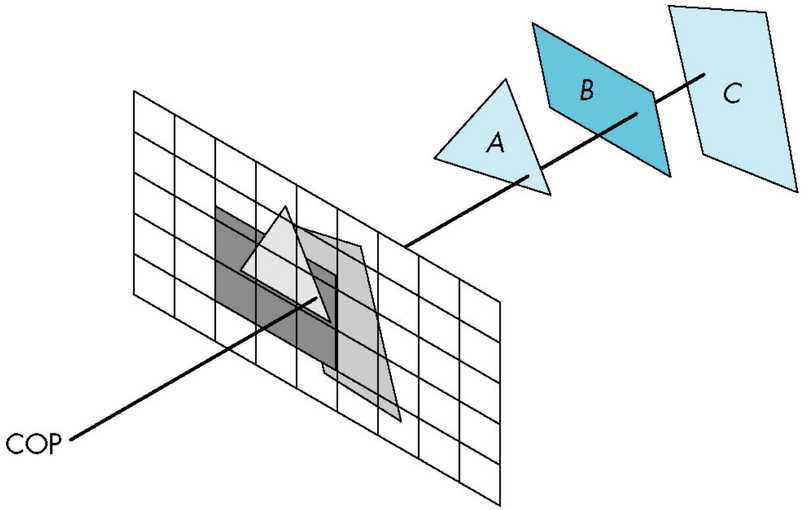
© [AW94] Figure 13.07

Visible Surface Determination

- VSD is eventually needed at the fragment level
  - results in a set of visible pixels for each graphics primitive
- Complex and potentially time-consuming task

Two basic approaches:

- Image-precision (image-space)
  - performed at a fixed resolution of the discrete rasterization
  - aliasing problems in visibility calculations
- Object-precision (object-space)
  - performed at the precision of the object definition
  - complex geometry processing operations



© [A00] Figure 06.42

Visible Surface Determination Approaches

1. For each pixel (image-space)
   - determine the surface that is intersected by the pixel projector (ray) that is closest and draw its color

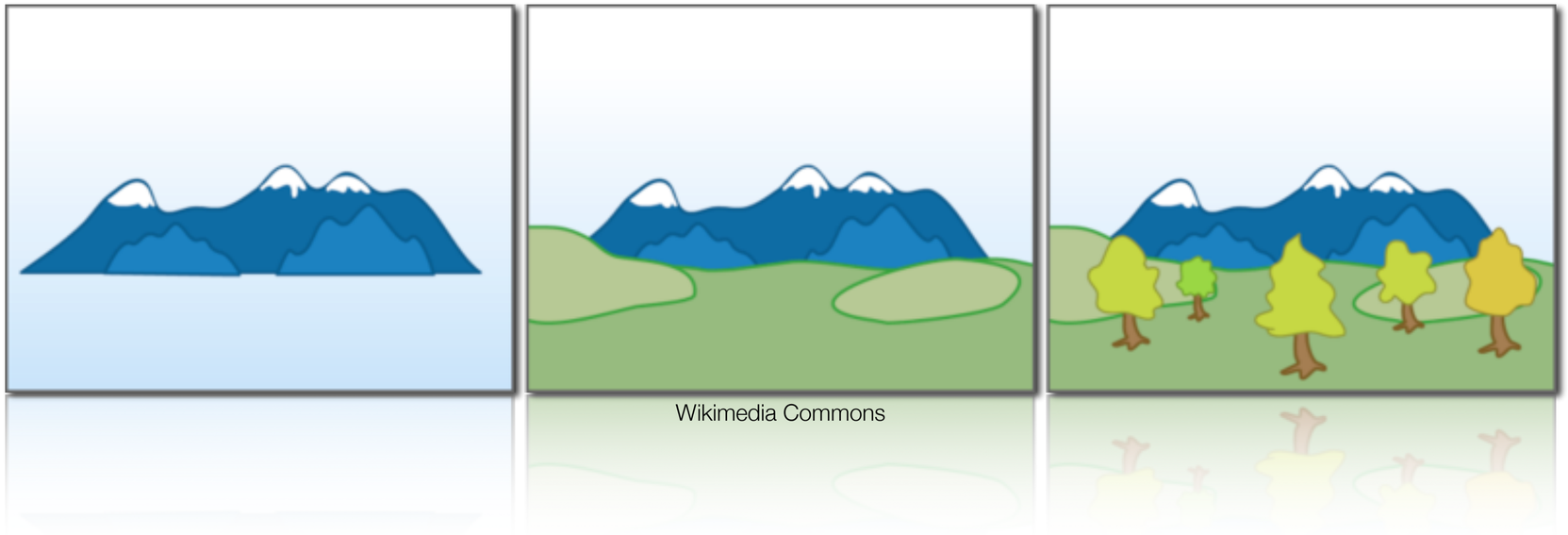
2. For every surface (object-space)
   - test against itself and all others for occlusion, determine the region that is visible and fill by color

- First approach generally needs $\mathrm{O}(np)$ time and second $\mathrm{O}(n^2)$ for n surfaces and p pixels
  - not clear which is best in general
  - even though for high-resolution displays with $n << p$ the approach may seem superior,
    its algorithmic complexity is higher → slower

Past and Now

- VSD problem has historically been addressed by clever and computationally efficient algorithms
  - ordering of geometric objects
  - recursive subdivision of object or screen space
- Modern practice relies mostly on simple and fast hardware accelerated solutions
  - fast fragment level determination of closest object
- … or raytracing
  - in contrast to the real-time rasterization pipeline
  - solves explicit ray-object intersections problems

Painter’s Algorithm

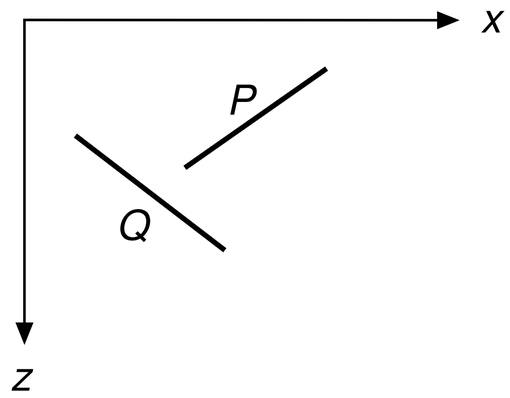


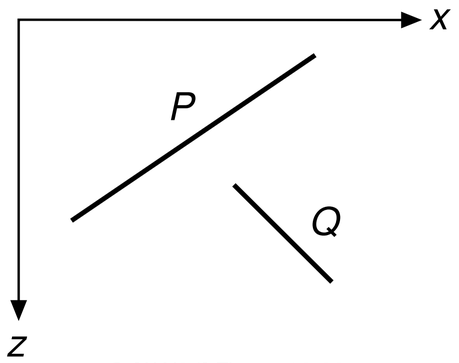
Wikimedia Commons

- Depth sorting of polygons followed by back-to-front drawing,
  i.e. with decreasing distance of polygon to viewpoint
  1. sort polygons (e.g. by their center of mass)
  2. resolve depth ambiguities
  3. scan convert polygons back-to-front (like painting on top of all previous ones)

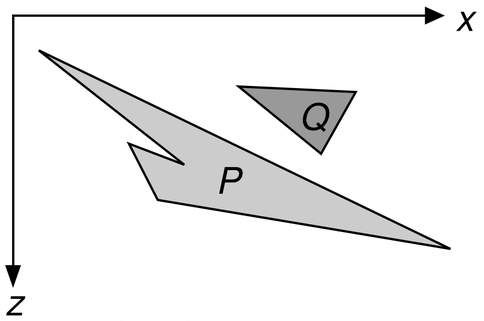
Polygon Sorting Order

- Painting order is clear if polygons have no depth-interval overlap along the z-axis
  - all vertices of one polygon are clearly in front of the other polygon's vertices
- Other configurations of polygons may still have a clear ordering
- If overlapping in z, P can safely be drawn before Q upon first true test of:
  1. do the polygons not overlap in the x coordinate?
  2. do the polygons not overlap in the y coordinate?
  3. is P completely on the opposite, far side of Q from the view point?
  4. is Q entirely on the same near side of P as the view point?
  5. do the projections of the two polygons not overlap?
- Invert order for tests 3 and 4





© [AW94] Figure 13.22

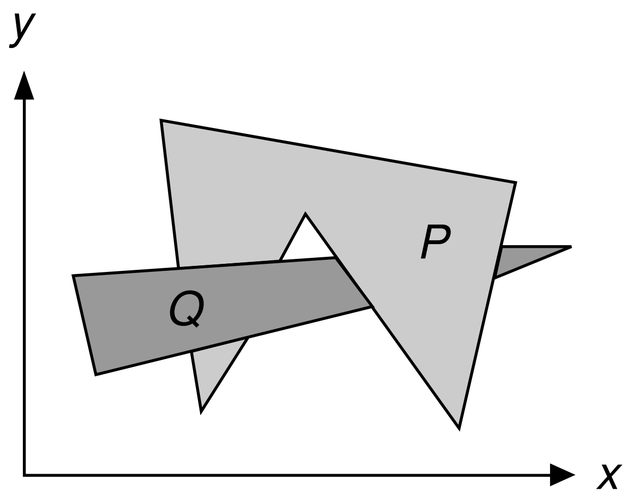


© [AW94] Figure 13.21a

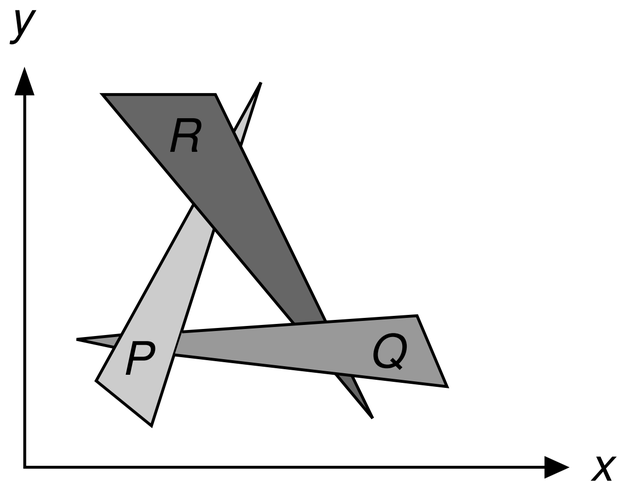
Depth Ambiguities

- If no simple ordering can be found, at least one polygon must be split into two parts
  - split by intersecting one polygon by the plane of the other

- Depth-cycles must be detected and resolved
  - mark visited polygons to detect a cycle
  - split at least one polygon to break a cycle



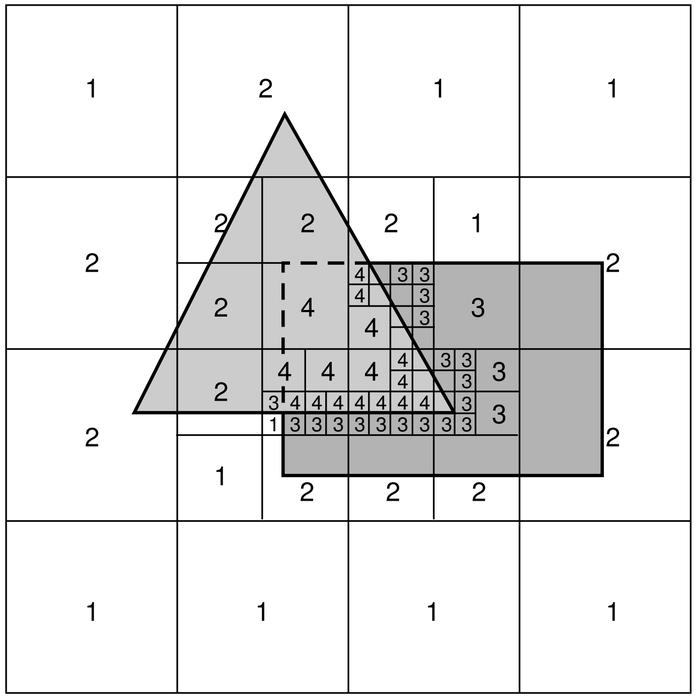
© [AW94] Figure 13.21b



© [AW94] Figure 13.21c

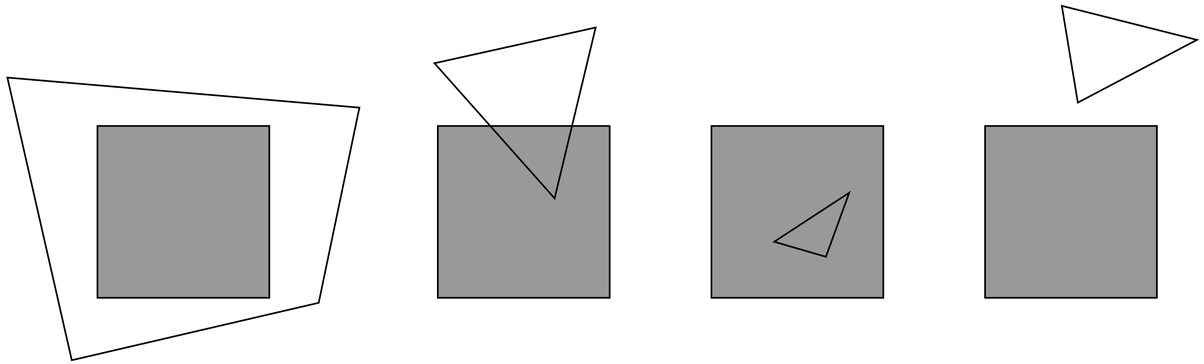
Warnock’s Algorithm

- Divide and conquer approach to the visibility problem based on spatial partitioning of the projection or image plane
  - recursively subdivide screen area into cells until a cell is easy to draw
- Quadtree subdivision into cells is performed until reaching the resolution of the raster image in the limit
  - fewer polygons overlap smaller quadtree cell regions
  - ultimately a decision on the fragment level can be reached



© [AW94] Figure 13.25

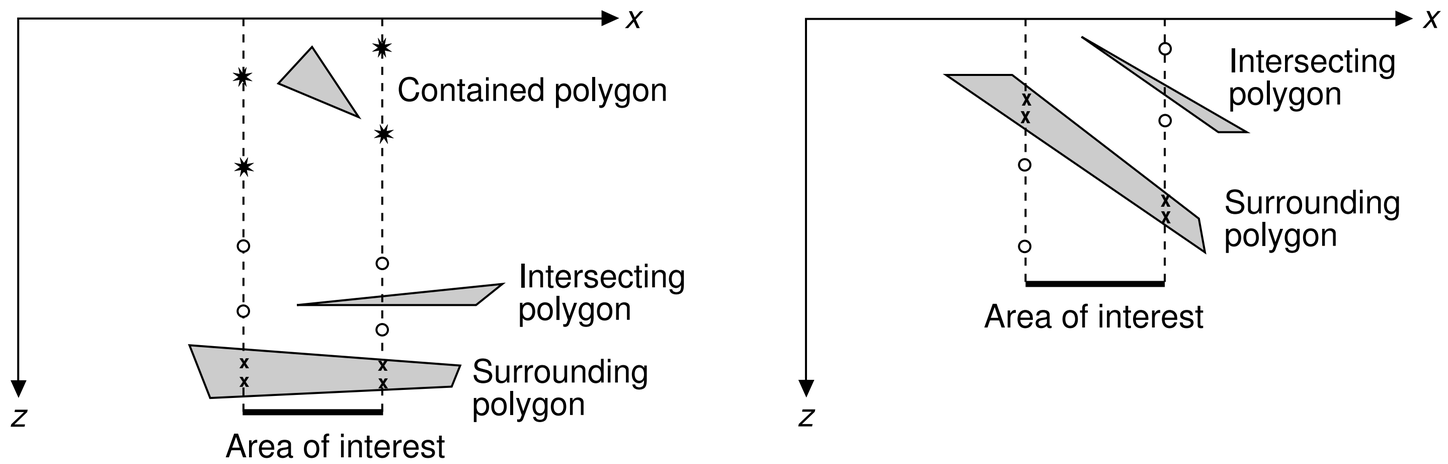
Polygon-Cell Overlap Tests



© [AW94] Figure 13.23

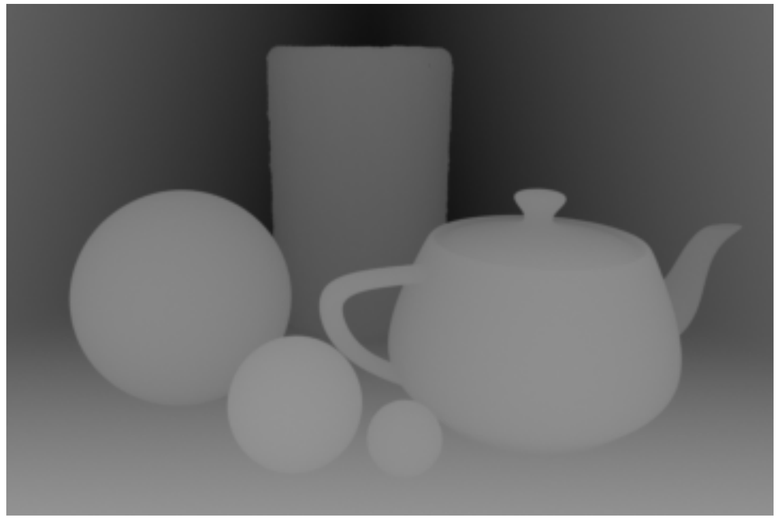
- Cell subdivision is based on 4 simple polygon-cell intersection cases
- Evaluation of four basic cell configurations:
  1. **all** polygons disjoint form the cell → leave background
  2. **one** intersecting or contained polygon → paint clipped polygon
  3. **one** single surrounding polygon → fill cell with polygon color
  4. **one** surrounding polygon in front of all others → fill cell with polygon color
- Else recursively subdivide cell and test

Surrounding Polygon Depth Test (Case 4)

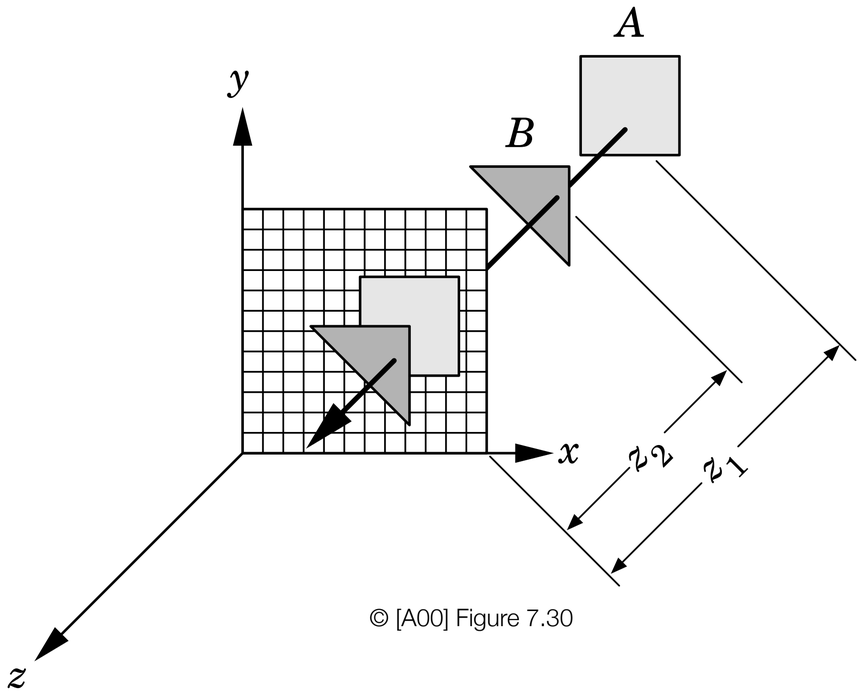


- Examine the z coordinates of all polygon planes at the corners of the cell
  - determine if one infinite plane is clearly in front of the others for this specific cell only
- If there is a surrounding polygon whose four corner z values are all closer to the viewer then every other polygon, then it is in front of all of them
  - evaluate all polygon plane equations for the z-value given all four corner x,y-values of the cell
- For fragment level cell, test is performed only at the center position

z-Buffer Algorithm



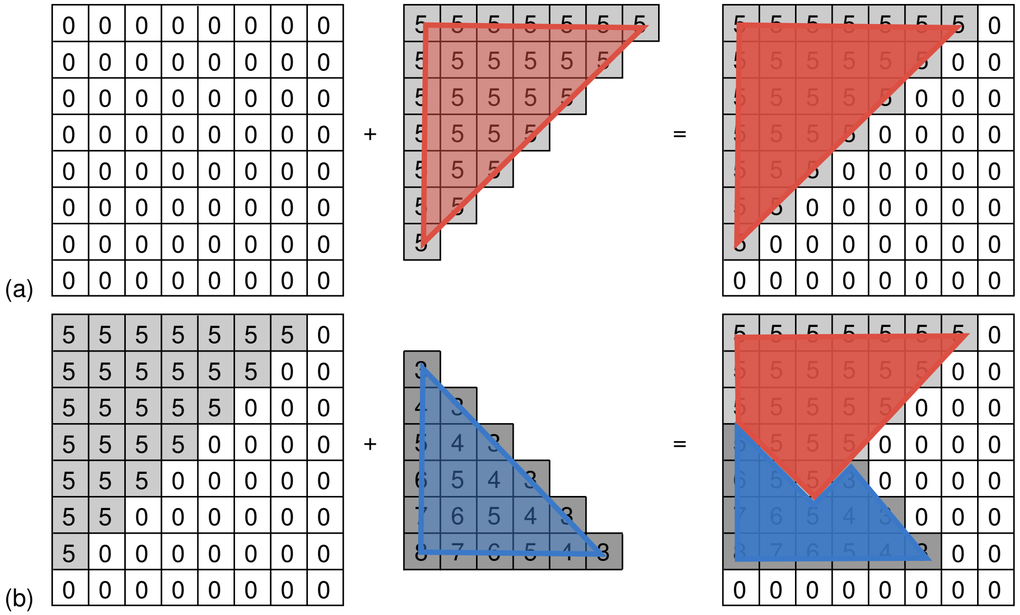
© Source: https://forum.processing.org/two/discussion/2153/how-to-render-z-buffer-depth-pass-image-of-a-3d-scene



© [A00] Figure 7.30

- During rasterization of each polygon, the fragment's depth value is written into the z-buffer, in addition to the color that is written into the frame buffer
  - z-buffer stores depth, the distance from the camera, in addition to color values per fragment
- Image-precision approach that determines the visibility per fragment using the values in the z-buffer
  - for each new fragment, simply compare the new fragment's z-value with the one previously stored in the z-buffer

z-Buffer Algorithm



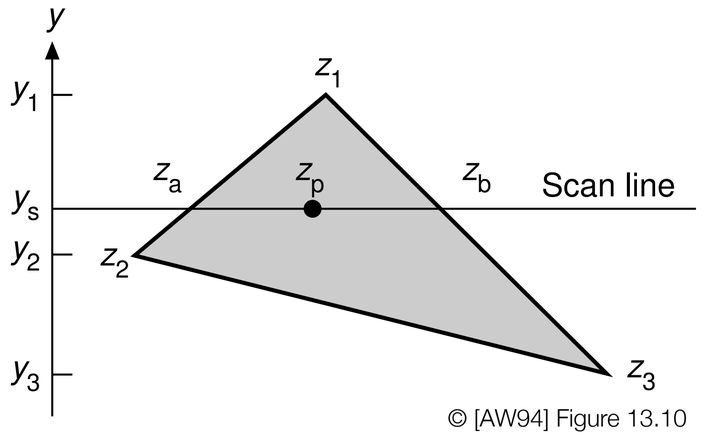
© [AW94] Figure 13.09

Example from Foley et al. book uses **larger values being closer to the viewer**.

- Initialize frame and depth-buffers to background color and maximal z-depth
- Foreach polygon
  - rasterize polygonal into fragments
  - foreach generated fragment
    - write fragment color and depth, if and only if the new depth is closer to the camera than the value stored in the z-buffer

Depth Interpolation

- Per-fragment depth calculation can be integrated with scan-conversion during the rasterization stage
  - exploit planarity of polygonal faces and depth-coherence on smooth surface patches
- Over a triangle, the z-coordinate can be linearly interpolated along the edges between their endpoints and across the scan-line between the two scan-line intersections
  - incremental calculation from one scan-line to the next



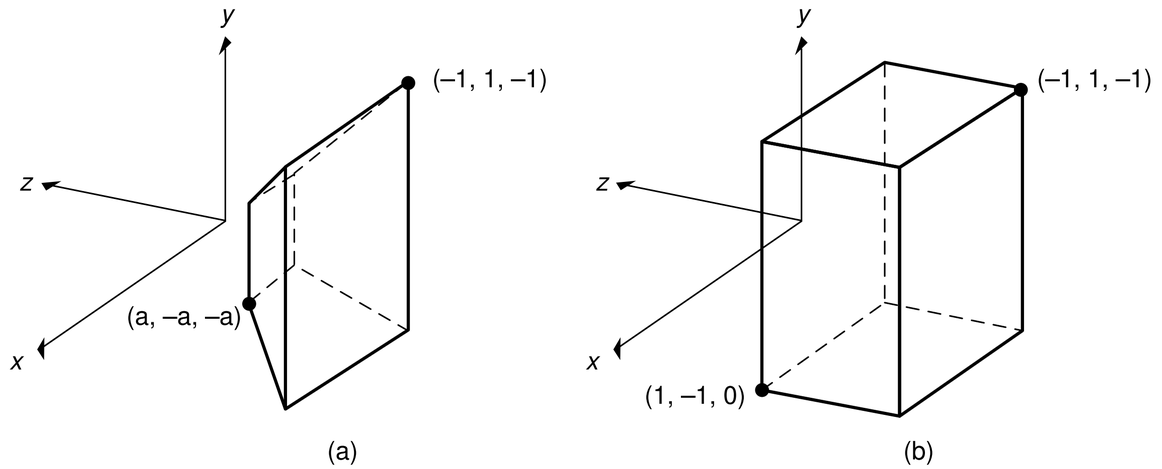
© [AW94] Figure 13.10

$$z_a = z_1 - (z_1 - z_2)\frac{y_1 - y_s}{y_1 - y_2}$$

$$z_b = z_1 - (z_1 - z_3)\frac{y_1 - y_s}{y_1 - y_3}$$

$$z_p = z_b - (z_b - z_a)\frac{x_b - x_p}{x_b - x_a}$$

z-Buffer Algorithm



© [AW94] Figure 13.02

- Operates effectively in normalized parallel-projection canonical view volume
  - pixel corresponds to projection ray parallel to the z-axis, hence z-value directly represents depth distance
- Polygons can be rasterized in arbitrary order
  - any pixel in frame buffer will be overwritten if new fragment is closer to viewpoint than current z-buffer depth value
  - z-buffer has to be initialized to the back clipping plane

Properties

- **image precision method** with fixed display and depth resolution
  - prone to aliasing problems and flickering effects from numerical inaccuracies
  - depth resolution has to match range of possible z-values
- **no presorting** required and easy to implement
  - polygons are drawn in their processing order
  - integrated with scan conversion, depth can be computed incrementally
- **no object comparisons**
  - one depth comparison per pixel and polygon containing it
  - processing depending on fill-rate, number of generated fragments
- **no explicit intersection tests**
  - visibility can be determined on any type of surface that can be rasterized
- **buffer records smallest z-depth** found so far for a given x,y
  - search over fixed x,y coordinate to find smallest z-depth
- **color does not have to be computed if fragment is not visible**
  - complex illumination and shading calculations can be avoided for hidden fragments (if surfaces are processed front-to-back)

Copyrights

- Many figures in these slides are copyright protected as indicated. Text and all other figures are copyright protected by Renato Pajarola.

- [SIG97] SIGGRAPH 97 Education Slide Set, Mapping Techniques, R.J. Wolfe.
- [AW94] Electronic Instructors Manual (EIM) for Introduction to Computer Graphics by James Foley, Andries van Dam, Steven Feiner, John Hughes, and Richard Phillips. Copyright Addison Wesley 1994.
- [A00] Electronic Book Support for Interactive Computer Graphics: A Top-Down Approach Using OpenGL by Edward Angel. Copyright E. Angel 2000. (http://www.cs.unm.edu/~angel/BOOK/SECOND_EDITION/)
- [AC97] Learning Rendering CD accompanying 3D Computer Graphics by Alan Watt. Copyright A. Watt and L. Cooper 1997. (Permission for copying and distribution without direct commercial advantage. To copy otherwise, or to republish, requires specific permission.)
- [S02] Electronic Figures for Fundamentals of Computer Graphics by Peter Shirley. Copyright P. Shirley 2002. (http://www.cs.utah.edu/~shirley/fcg/figures/)
- [N05] GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation. (http://http.developer.nvidia.com/GPUGems/gpugems_ch09.html)